

1 Basic R

Learning Objectives:

The journey of R scripting starts with a basic understanding of R objects and commands. After finishing this chapter, you should be able to:

- Define R variables.
- Know the differences between an atomic variable and an object variable.
- Store results in variables through the assignment operator.
- Perform arithmetic operations.
- Handle composite objects: vectors, and data frames.
- Define categorical variables.
- Calculate descriptive and summary statistics.
- Utilize inspection functions to figure out basic information of data.
- Format and print the contents of variables to the console.
- Use `seq()` and `with()` functions.

As R scripting is a hands-on topic, I encourage you to open RStudio alongside this book and try out the code to maximize understanding. Let's get started.

1.1 R Variables

An R variable is a named location in the computer's memory that stores some values. Imagine it as a labeled (name) shoebox (memory location) keeping a pair sneakers (value).

Two rules determines how an R variable be named:

1. It must begin with a letter (A-Z or a-z), a dot “`.`”, or an underscore “`_`”, and
2. Subsequent characters can be any combinations of letters, digits (0-9), “`.`”, or “`_`”.

Note that:

- A variable name can be as short as one character to way longer than a normal person wishes (10,000 characters!).

- My suggestion is to give variables meaningful names that can help a bunch for you or someone to comprehend the the code. For instance, in a problem about heart rate, use `zone2` instead of `z` or even worse `x`.
- Be careful that variable names are case-sensitive, e.g., `volume` and `Volume` are different in the eyes of R.
- Avoid naming variables with names that have been reserved by R (<https://rdr.io/r/base/Reserved.html>) or names already used to name R's built-in functions, e.g., `c`, and `head`. For the latter, unfortunately, R is permissive for misnaming variables by remaining silent, costing time to identify the issue. Therefore, the rule of thumb is to exercise self-discipline not to choose R function names as variable names.

1.2 R Objects

Every variable is an R object. In scripting, it is important to know the type(s) of value(s) being stored in a variable, as the types dictate the legitimate operations that act on the variable. Let's begin with data types.

1.2.1 Data Types

R supports six data types:

- Double: decimal numbers, e.g., 5.6, -4.3
- Integer: whole numbers, e.g., -10, 56
- Character: they contain text, e.g., "ACGT", "Hello World"
- Logical: `TRUE` or `FALSE`
- Complex*: Complex numbers containing real and imaginary components, e.g., 5+2i
- Raw*: Bare bytes of data.

* We will only discuss the first four types in this book, you can ignore complex and raw types.

1.2.2 Atomic and Composite Variables

From a practical perspective, R variables can be viewed as **atomic variables**, and **composite variables**. An atomic variable contains a single value, and a composite variable consists of a collection of values. Let us begin with two atomic variables: `numeric`, and `character`.

1.2.2.1 Numeric Variables

An atomic numeric variable holds a decimal number (`double`) or a whole number (`integer`). Numbers can be represented in scientific notation. Here are some examples:

```
# Decimal
height <- 5.6

# Whole number (Integer)
```

```
sample_size <- 15L

# Scientific notation
avogadro_number <- 6.023e23
```

`<-` is the assignment operator. It looks like a left-pointing arrow, depicting the transfer of value from the right-hand side to the left-hand side. Upon execution, the resultant value of the expression on the right-hand side, if no error, is stored in the named variable on the left-hand side. In RStudio, you can see the variable name and its associated value under the **Environment** tab, which usually occupies the top right quadrant of the RStudio desktop.

Note that R supports another assignment operator: `=`. You can use `<-` and `=` interchangeably. I prefer `<-` to `=`, as `=` can easily confuse with the equality checking operator `==`.

A few words about the scientific notation. It comprises three parts: the coefficient, a constant literal `e`, and the exponent. The `e` refers to the base `10`. For example, the R's scientific notation of 6.023×10^{23} is `6.023e23`, where the coefficient and the exponent are `6.023` and `23`, respectively. Note that you must not include spaces in a scientific notation, e.g., `6.023e 23`, `6.023 e23`, or `6.023 e 23` will return errors.

Arithmetic Operators

You can apply arithmetic operations on numeric variables. Here are the arithmetic operators: `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `**` (exponentiation), and `^` (exponentiation). The last two exponentiation operators perform exactly the same function. Here are some examples:

```
radius <- 2.5
diameter <- 2*pi*radius
pi <- 3.1416
area <- pi*radius**2
volume <- (4/3)*pi*radius^3
```

1.2.2.2 Character Variables

R also support text values, which is in `character` data type. To define a character value, you need to use a pair of quotes to embrace the text. Either a pair of single (`'`) or double quotes (`"`) works. E.g.,

```
course <- 'BIOL265'
```

To figure out whether a variable's type is numeric, character, or something else, use the `typeof()` function,

```
typeof(course)
```

```
[1] "character"
```

```
typeof(area)
```

```
[1] "double"
```

Obviously, it does not make sense to do math with character type. E.g.,

```
course*4
```

```
Error in `course * 4`:  
! non-numeric argument to binary operator
```

You can use `nchar()` function to figure out the number of characters, including spaces and punctuations, of a character variable. E.g.,

```
nchar(course)
```

```
[1] 7
```

1.2.3 Vectors

One common composite variable in R is **vectors**. It is an ordered sequence of items that can be processed collectively as a whole, or individually. A vector can be viewed pictorially as a wagon train in which carriages are coupled one after the other. E.g.,

```
# A simple numeric vector for the days in each month  
days_of_month <- c(31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)
```

`days_of_month` is a **vector** object in R. `c()` is the function that **combines or collates** several objects into a vector object. Items are indexed or numbered from 1 to the total number of items in the vector. You can extract a particular item by specifying the index within a pair of square-brackets. E.g.,

```
# Days in February  
print(days_of_month[2])
```

```
[1] 28
```

The `[2]` means the 2nd element of a vector. We will discuss more about indexing below.

Here is an example of a character vector:

```
nucleotides <- c('A', 'C', 'G', 'T')
```

Noteworthy that all items held inside a vector must have the same data type. In other words, vectors are always **homogeneous**. For instance, `days_of_month` contains all numeric values, and `nucleotides` contains all character values. If you try to combine objects of different data types into a vector, R will automatically convert them to character type, the lowest denominator of data types. In the example below, even the first and third values are entered as numeric, R coerces them into text `"1"` and `"0"`.

```
a_vector <- c(1, "success", 0, "fail")  
a_vector
```

```
[1] "1"      "success" "0"      "fail"
```

One of the R features that makes arithmetic operations on vectors really convenient is **vectorization**. For instance, if you want to standardize the values of a sample such that the sample mean and sample variance after standardization become 0 and 1, respectively. The formula for standardization is $z_i = \frac{(x_i - \bar{x})}{s}$, where $\bar{x}$ and s are the sample mean and sample standard deviation, respectively. Here's an example

```
heights <- c(5.5, 6.0, 6.1, 5.7, 5.9, 5.6)
x_bar <- mean(heights)
s <- sd(heights)
std_heights <- (heights - x_bar)/s
std_heights
```

```
[1] -1.2677314  0.8451543  1.2677314 -0.4225771  0.4225771 -0.8451543
```

`mean()` and `sd()` are built-in functions that calculate the average and standard deviation of a sample, respectively.

The convenience is that you don't have to specify how to subtract and divide each height individually. Instead, R recognizes the expression `(heights-x_bar)` is to subtract an atomic `x_bar` (scalar) from a vector `heights`. Under the vectorization mechanism, it will subtract `x_bar` from every height in the vector `heights`, similarly for the division by `s`.

vector is only one of the many types of objects supported by R. Data frame is another commonly used object types, which will be discussed later.

1.2.4 Factors/Categorical Variables

In R, a **factor** refers to a categorical variable in statistics. A categorical variable is further classified into **nominal** or **ordinal**. A nominal variable has no intrinsic order. Use the `factor()` function to define a categorical variable in R. E.g., a sample of colors is stored in a vector, and you want to define it as a nominal categorical variable.

```
colors <- c("Red","Blue","Blue","Red","Green","Green","Blue","Green","Red","Blue")
colors <- factor(colors, levels=c('Red','Blue','Green'))
colors
```

```
[1] Red  Blue Blue Red  Green Green Blue  Green Red   Blue
Levels: Red Blue Green
```

Categorical variables with intrinsic order are called **ordinal** variables. E.g., the stages of hypertension.

```
patients <- c("Stage 2","Hyperintensive","Stage 2","Hyperintensive","elevated","Stage 1",
patients <- factor(patients,
                    levels=c('normal','elevated','Stage 1','Stage 2','Hyperintensive'),
                    ordered = TRUE)
patients
```

```
[1] Stage 2      Hyperintensive Stage 2      Hyperintensive elevated
[6] Stage 2      elevated      Stage 1
Levels: normal < elevated < Stage 1 < Stage 2 < Hyperintensive
```

As you can see above, the last line (**Levels**) shows the hypertension levels in ascending order and levels are connected by a less than sign (**<**).

1.2.5 Data Frames

Besides **vector** , another common R object is **data frame** . Almost all the datasets used for data analysis are organized in data frames. Simply speaking, a data frame is a **tabular** or spreadsheet-like object where values are organized in rows and columns. Each row represents a sample and each column represents a variable. Rows are expected to have equal length, i.e., the same number of columns. If not, R will either complain or pad the missing columns with a special value **NA** (it stands for not available). Let's take a look at an R's built-in data frame **iris** :

```
# Show the first five rows
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

You can peek into the bottom five rows by the **tail()** function.

```
tail(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
145	6.7	3.3	5.7	2.5	virginica
146	6.7	3.0	5.2	2.3	virginica
147	6.3	2.5	5.0	1.9	virginica
148	6.5	3.0	5.2	2.0	virginica
149	6.2	3.4	5.4	2.3	virginica
150	5.9	3.0	5.1	1.8	virginica

As you can see above, the **iris** data frame consists of five variables (columns). Here are the R functions to show the shape of a data frame:

```
# the dimension (row x column) of a data frame.
dim(iris)
```

```
[1] 150  5
```

```
# Only the number of rows
nrow(iris)
```

```
[1] 150
```

```
# Only the number of columns  
ncol(iris)
```

```
[1] 5
```

So, we know that `iris` contains 150 rows (flower samples) and five columns.

1.2.5.1 Accessing Rows and Columns of a Data Frame

Each element in a data frame can be referenced by row and column indexes like a X-Y Cartesian system. Rows are automatically indexed (numbered), where the 1st row is index 1, the 2nd row is index 2, and the index of the last row is the total number of rows, similarly for columns.

The syntax of accessing individual elements by indexes is `<data frame name>[row, column]`. E.g., the cell at the 5th row and 3rd column of the `iris` is:

```
iris[5,3]
```

```
[1] 1.4
```

If the column index is ignored, all columns of the specified row(s) are returned as shown in Figure 1.1. For example, show all columns of the 3rd row by:

```
# Specify the row number only and leave out the column index.  
iris[3,]
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
3	4.7	3.2	1.3	0.2	setosa

Column Index

Row Index	1	2	3	4	5
	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa

iris[3,]

iris[5,3] or iris[5,'Petal.Length']

iris[,5] or iris\$Species

Figure 1.1: Accessing rows and columns of a data frame.

Similarly, if the row index is skipped, all rows of the specified column(s) are returned. For example,

```
# Specify only the column index.
head(iris[,5])
```

```
[1] setosa setosa setosa setosa setosa setosa
Levels: setosa versicolor virginica
```

Note that the `head()` above is used to limit the number rows displayed by avoiding listing page after page of data rows.

You are not limited to display one row or one column using this indexing scheme. For example, show to 110th and 34th rows and 2nd and 5th columns,

```
iris[c(110,34), c(2,5)]
```

```
   Sepal.Width  Species
110         3.6 virginica
 34         4.2   setosa
```

As columns are often named¹, columns can be referenced by their names as well ([Figure 1.1](#)). E.g., the `Petal.Length` of the 5th row:

```
iris[5,'Petal.Length']
```

```
[1] 1.4
```

To access more than one column and row, pack the row indexes and column names in vectors by the `c(...)` function, which combines or collates several items into a vector object (see [Section 1.2.3](#)). For example, show the `Petal.Length` and `Sepal.Length` of the 123th and 96th rows:

```
iris[c(123,96),c('Petal.Length', 'Sepal.Length')]
```

	Petal.Length	Sepal.Length
123	6.7	7.7
96	4.2	5.7

If your task is to extract a single column, R provides an alternative way to access a column by its name. The syntax is `<data frame name>$<column name>`². For example, `iris$Species`, which is equivalent of `iris[,5]`,

```
head(iris$Species)
```

```
[1] setosa setosa setosa setosa setosa setosa
Levels: setosa versicolor virginica
```

We have covered several ways to access rows, columns, or both using indexes and column names. There are additional methods to select rows and columns by conditions. More of this topic will be discussed in [Chapter 2](#).

1.2.5.2 Create a Data Frame

We can use the `data.frame()` function with the concept of vector to create a dataset in data frame format. For example, a study collected 10 fish with their sex and age. To create the data frame, vectors must be created to represent each variable (column). Here's the R code:

```
sex <- c('F','M','M','F','F','F','M','M','M','M')
age <- c(1,3,2,3,1,1,3,3,1,1)
fish <- data.frame(sex,age)
fish
```

	sex	age
1	F	1
2	M	3
3	M	2
4	F	3
5	F	1
6	F	1
7	M	3
8	M	3
9	M	1
10	M	1

You can use the `summary()` function to examine the summary statistics of the variables.

```
summary(fish)
```

```

      sex      age
Length  :10   Min.   :1.0
N.unique : 2   1st Qu.:1.0
N.blank  : 0   Median :1.5
Min.nchar: 1   Mean    :1.9
Max.nchar: 1   3rd Qu.:3.0
           Max.    :3.0

```

Find out more information about `summary()` function below [Section 1.3.3](#).

It is almost done. There is one nuance. In the current form, `sex` is a character variable so its summary statistics are a bit strange. Most often, variable likes that should be categorical/nominal. Therefore, we need to convert `sex` into a `factor` [Section 1.2.4](#). Here's the revised code:

```

sex <- c('F','M','M','F','F','F','M','M','M','M')
sex <- factor(sex, levels=c('F','M'))
age <- c(1,3,2,3,1,1,3,3,1,1)
fish <- data.frame(sex,age)
fish

```

```

  sex age
1   F   1
2   M   3
3   M   2
4   F   3
5   F   1
6   F   1
7   M   3
8   M   3
9   M   1
10  M   1

```

See the differences for `sex` after revision.

```
summary(fish)
```

```

sex      age
F:4   Min.   :1.0
M:6   1st Qu.:1.0
      Median :1.5
      Mean    :1.9
      3rd Qu.:3.0
      Max.    :3.0

```

1.3 Descriptive Statistics

1.3.1 Non-robust Statistics

Sample mean, variance, and standard deviation are non-robust statistics, as they are sensitive to outliers (extreme values).

```
# A sample of pumpkins' weights  
pumpkins <- c(6.85,6.56,6.23,7.55,4.45,10.14,3.52,6.62,3.76,7.62)
```

Sample Mean

```
# Sample mean or the average weight  
mean(pumpkins)
```

```
[1] 6.33
```

Sample Variance

```
# Sample variance  
var(pumpkins)
```

```
[1] 4.013489
```

Sample Standard Deviation

```
# Sample standard deviation  
sd(pumpkins)
```

```
[1] 2.003369
```

By the way, the square root of the sample variance $\sqrt{\text{variance}}$ is the standard deviation. See below:

```
sqrt(var(pumpkins))
```

```
[1] 2.003369
```

1.3.2 Robust Statistics

Median and quantile are called robust statistics, as they impede outliers' influence.

Median

```
# A sample of pumpkins' weights  
pumpkins <- c(6.85,6.56,6.23,7.55,4.45,10.14,3.52,6.62,3.76,7.62)
```

```
median(pumpkins)
```

```
[1] 6.59
```

Quantile

The default quantile interval, as the name suggests, is 25%. Note that the R function name is `quantile()`, not `quartile()`, as users can override the default 25% interval.

```
quantile(pumpkins)
```

0%	25%	50%	75%	100%
3.520	4.895	6.590	7.375	10.140

Instead of the default 25% interval, you can specify your preferred quantile(s) and they don't have to be equally divided. Here are some examples:

```
quantile(pumpkins, probs = c(0.1,0.9))
```

10%	90%
3.736	7.872

Or, a specific quantile.

```
quantile(pumpkins, probs = c(0.05))
```

5%
3.628

```
quantile(pumpkins, probs = c(0.95))
```

95%
9.006

Inter-Quartile Region (IQR)

Here is the definition of IQR:

$$IQR = Q3 - Q1$$

```
IQR(pumpkins)
```

```
[1] 2.48
```

IQR is used to determine the upper and lower whiskers in a boxplot. Data points below or above the lower or upper whisker are considered as outliers.

$$\text{Lower whisker} = Q1 - 1.5 \times IQR$$

$$\text{Upper whisker} = Q3 + 1.5 \times IQR$$

1.3.3 summary() Function

A data frame may consists of many variables of different types in various ranges. Therefore, it makes a lot of sense to take a glimpse of all variables before analysis. It can be done by using the `summary()` function to reveal:

1. The variable type of each column.
2. If the column is numeric, a summary of numeric values.
3. If the column is categorical (factor), what are the levels and order, if it is ordinal.

Let's see the summary of `iris`:

```
summary(iris)
```

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
Min. :4.300	Min. :2.000	Min. :1.000	Min. :0.100
1st Qu.:5.100	1st Qu.:2.800	1st Qu.:1.600	1st Qu.:0.300
Median :5.800	Median :3.000	Median :4.350	Median :1.300
Mean :5.843	Mean :3.057	Mean :3.758	Mean :1.199
3rd Qu.:6.400	3rd Qu.:3.300	3rd Qu.:5.100	3rd Qu.:1.800
Max. :7.900	Max. :4.400	Max. :6.900	Max. :2.500

Species	
setosa	:50
versicolor	:50
virginica	:50

The summary will show the range (min. and max.) and quantiles of a numeric column. For a categorical variable (factor), it will show the levels with the number of samples (rows) found for each level, e.g., `Species` is a nominal variable, with 50 samples per species.

1.4 Utility Functions

1.4.1 Inspection Functions

Before diving into data analysis, we often want to know some basic information about it, such as the number of samples, what is the range of a numeric variable, etc. Here we will discuss a few R functions to do that, and we coin them inspection functions.

We will use a sample of pumpkin's weights for illustration.

```
# A sample of pumpkins' weights
pumpkins <- c(6.85, 6.56, 6.23, 7.55, 4.45, 10.14, 3.52, 6.62, 3.76, 7.62)
```

The `length()` shows the number of items in a vector.

```
length(pumpkins)
```

```
[1] 10
```

To be clear, `length()` counts the number of items in an R composite object; the number of items in a vector or the number of columns in a data frame. **If the object is atomic, the function will return 1.** For example, in the following example, `dna` is an atomic, character variable. The function will return 1, not the number of characters.

```
dna <- 'ACGTAGC'  
length(dna)
```

```
[1] 1
```

To count the number of characters in a character variable, use the `nchar()` function.

```
nchar(dna)
```

```
[1] 7
```

The `range()` prints the minimum and maximum values of an object ³.

```
range(pumpkins)
```

```
[1] 3.52 10.14
```

If you are interested in either the minimum or maximum, here are the corresponding functions:

```
# The lightest  
min(pumpkins)
```

```
[1] 3.52
```

```
# The heaviest  
max(pumpkins)
```

```
[1] 10.14
```

1.4.2 Printing and Formatting

R provides a function called `print()` that prints text to the console. E.g.,

```
print("Hello World!")
```

```
[1] "Hello World!"
```

```
radius <- 2.5  
diameter <- 2*pi*radius
```

```
print(diameter)
```

```
[1] 15.708
```

Suppose you want to improve the printout by making it more readable, say "The diameter of a circle with radius 2.5 cm is 15.70796." In such a case, you need the format function `paste()`. Here's how to use it:

```
radius <- 2.5
diameter <- 2*pi*radius
print(paste("The diameter of a circle with radius",radius,"cm is",diameter,"."))
```

```
[1] "The diameter of a circle with radius 2.5 cm is 15.708 ."
```

The above printout is almost perfect. If you're picky, you don't like the extra space separating the diameter and the period in the above message, you can use `paste()`'s cousin function `paste0()`. The difference between the two is that `paste0()` will not automatically add an extra space between each component given to the function. Below is the previous print statement with `paste0()`:

```
print(paste0("The diameter of a circle with radius",radius,"cm is",diameter,"."))
```

```
[1] "The diameter of a circle with radius2.5cm is15.708."
```

As you can see, there is no more space between the diameter and the period. But it created other issues; there is no space between "The diameter of a circle with radius" and the actual radius as well. So, when you use `paste0()`, you have to take care of the space separation yourselves. Here is the final version for using `paste0()`:

```
print(paste0("The diameter of a circle with radius ",radius," cm is ",diameter,"."))
```

```
[1] "The diameter of a circle with radius 2.5 cm is 15.708."
```

1.4.3 `seq()` - create a series of numbers

There are situations where we need a series of equal-interval numbers. For example, override the default breaks - x-ticks - of a histogram, a vector containing all possible outcomes of tossing two dice, i.e, from 2 to 12.

In R, the `seq()` function generates a list of equal-interval numbers, whole and decimal. `seq()` takes the following parameters:

1. `from = 1`, the default is 1.
2. `to = 1`, the default is 1.
3. `by = 1`, the size of the interval between two adjacent numbers. It takes integer and decimal numbers. If it is negative, the series is decreasing.
4. `length.out` specifies how many numbers are returned. This option is an alternative to `by` when you care only about how many numbers are generated instead of the interval between them. See the example below.

5. `along.with`, it generates the same number of numbers according to what is provided to the `along.with` parameter.

Examples

By default, `seq()` begins with 1 and has an interval of 1. E.g., generate 10 integers starting with 1.

```
seq(10)
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

Starting from and ending with.

```
seq(-10, 10)
```

```
[1] -10 -9 -8 -7 -6 -5 -4 -3 -2 -1 0 1 2 3 4 5 6 7 8
[20] 9 10
```

Specify the interval.

```
seq(-10, 10, by=2)
```

```
[1] -10 -8 -6 -4 -2 0 2 4 6 8 10
```

A decreasing sequence.

```
seq(10, by=-1)
```

```
[1] 10 9 8 7 6 5 4 3 2 1
```

Just specify the desired numbers to be generated. As you can see, the interval is pretty precise.

```
seq(-5, 7, length.out=20)
```

```
[1] -5.00000000 -4.36842105 -3.73684211 -3.10526316 -2.47368421 -1.84210526
[7] -1.21052632 -0.57894737 0.05263158 0.68421053 1.31578947 1.94736842
[13] 2.57894737 3.21052632 3.84210526 4.47368421 5.10526316 5.73684211
[19] 6.36842105 7.00000000
```

Generate a sequence that matches the same number in another sequence.

```
x <- seq(1, 5)
cat("x =", x, "\n")
```

```
x = 1 2 3 4 5
```

```
y <- seq(-5, along.with=x)
```

```
cat("y =",y,"\n")
```

```
y = -5 -4 -3 -2 -1
```

Note: What is the difference between `cat(...)` and `print(paste(...))` you saw above? The main difference is that `print(paste(...))` creates an R object (a text) and then prints it, while `cat(...)` directly writes text to the console without creating a formal R object. See example below you will understand the nuance.

```
msg1 <- print(paste("Hello","World"))
```

```
[1] "Hello World"
```

```
print(msg1)
```

```
[1] "Hello World"
```

```
msg2 <- cat("Hello","World","\n")
```

```
Hello World
```

```
print(msg2)
```

```
NULL
```

Is it really important? No, unless you're a R developer.

1.4.4 `with()`

The function set up the context of a data frame so it spares you from using the `$` to access the columns of a data frame, as it often leads to longer commands, which makes harder to comprehend and troubleshooting. It is also commonly used together with a function applied to one or more columns. E.g., the median of `Petal.Width`. It can be done in two ways:

```
# Use $  
median(iris$Petal.Width)
```

```
[1] 1.3
```

```
# Or, use with()  
with(iris, median(Petal.Width))
```

```
[1] 1.3
```

For the straightforward example above, you do not see the value of `with()`. Let's consider a more complicated row selection example. Find `Sepal.Length` in `iris` where its `Petal.Length` is less than 4 and `Petal.Width` is greater than 1. The `$`-way is:

```
iris$Sepal.Length[iris$Petal.Length<4 & iris$Petal.Width>1]
```

```
[1] 5.2 5.6 5.6 5.5 5.8 5.1
```

Here's the `with()` version that saves typing `iris` repeatedly.

```
with(iris, Sepal.Length[Petal.Length<4 & Petal.Width>1])
```

```
[1] 5.2 5.6 5.6 5.5 5.8 5.1
```

In short, `with()` is English, so it improves the readability of R statements. Second, it saves some typing of the data frame's name. I recommend using `with()` whenever possible.

1.5 Summary

That's a bare minimum of R scripting required for data analysis. In the next chapter, we will focus on operations applied to data frames. Before you leave, it is useful to recap the R functions discussed in this chapter:

A Summary of R Operators and Functions

<code><-</code>	<code>head()</code>	<code>sd()</code>	<code>+</code>
<code>c()</code>	<code>tail()</code>	<code>range()</code>	<code>-</code>
<code>print()</code>	<code>dim()</code>	<code>min()</code>	<code>*</code>
<code>paste()</code>	<code>nrow()</code>	<code>max()</code>	<code>/</code>
<code>paste0()</code>	<code>ncol()</code>	<code>median()</code>	<code>**</code>
<code>cat()</code>	<code>data.frame()</code>	<code>quantile()</code>	<code>^</code>
<code>typeof()</code>	<code>summary()</code>	<code>IQR()</code>	<code>==</code>
<code>nchar()</code>	<code>mean()</code>	<code>seq()</code>	<code><-</code>
<code>factor()</code>	<code>var()</code>	<code>with()</code>	<code>c()</code>

1. If rows are named, rows can be referenced by their names. ↩
2. The `$` method only works for a single column. ↩
3. Note that `range()` works for non-numeric vector, but we will skip such special operation. ↩